Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

ОТЧЕТ

по лабораторной работе № 1

по курсу «Анализ алгоритмов»

на тему: «Сравнительный анализ ресурсной эффективности алгоритмов»

Студент ИУ7-52Б _____ Насута К. С.
(Группа) (Подпись, дата) (И. О. Фамилия)

Преподаватель _____ Строганов Д. В.
(Подпись, дата) (И. О. Фамилия)

2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **3**

1 Аналитическая часть **4**

1.1 Постановка задачи 4

2 Конструкторская часть **5**

2.1 Разработка алгоритмов 5

2.1.1 Линейный поиск 5

2.1.2 Бинарный поиск, нерекурсивная реализация 6

2.1.3 Бинарный поиск, рекурсивная реализация 10

2.2 Требования к программному обеспечению 11

3 Технологическая часть **12**

3.1 Средства реализации 12

3.2 Реализация алгоритмов 12

3.3 Тестирование реализации алгоритмов 12

4 Исследовательская часть **14**

4.1 Оценка затрачиваемой памяти 14

4.2 Оценка временной эффективности 15

4.2.1 Линейный поиск 15

4.2.2 Бинарный поиск 15

4.3 Результаты сравнительного анализа 16

4.4 Выводы 17

ЗАКЛЮЧЕНИЕ **18**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **19**

ПРИЛОЖЕНИЕ А **20**

ВВЕДЕНИЕ

Целью настоящей лабораторной работы является выполнение сравнительного анализа ресурсной эффективности алгоритмов.

Задачи работы:

— описать решаемую задачу согласно варианту;

— разработать рекурсивный и нерекурсивные алгоритмы решения задачи, согласно варианту;

— описать разработанные алгоритмы схемами алгоритмов;

— описать требования к программному обеспечению;

— описать средства реализации алгоритмов;

— реализовать разработанные алгоритмы;

— выполнить тестирование реализации алгоритмов;

— выполнить теоретическую оценку затрачиваемой реализацией каждого алгоритма памяти (для рекурсивного алгоритма — на материале анализа высоты дерева рекурсивных вызовов и оценки затрачиваемой на один вызов функции памяти);

— оценить временную эффективность алгоритмов или их реализаций в лучшем и худшем случае в терминах числа сравнений искомой величины с другими величинами;

— представить результаты сравнительного анализа в формате таблицы;

— сделать выводы из сравнительного анализа реализации разработанных алгоритмов решения одной и той же задачи, заданной вариантом, по критериям временной и ёмкостной эффективности, а также дополнительных требований, налагаемых алгоритмом.

1 Аналитическая часть

1.1 Постановка задачи

Дан массив A из N целых чисел; для заданного значения x требуется определить, содержится ли x в массиве A, и если содержится — вернуть его индекс в массиве.

Требуется разработать следующие алгоритмы:

— линейный поиск

— бинарный поиск, нерекурсивная реализация, вариант 1

— бинарный поиск, нерекурсивная реализация, вариант 2

— бинарный поиск, рекурсивная реализация

Сравнение нерекурсивных реализаций бинарного поиска между собой и с его рекурсивной реализацией является предметом анализа в настоящей работе.

Входные данные: натуральное число N, N целых чисел, образующих массив A, и искомое значение $x \in \mathbb{Z}$. Выходные данные: индекс элемента, равного x, либо -1, если элемент отсутствует.

2 Конструкторская часть

2.1 Разработка алгоритмов

Разработаны следующие алгоритмы: линейный поиск, нерекурсивный бинарный поиск (в двух вариантах) и рекурсивный бинарный поиск [1].

2.1.1 Линейный поиск

Нерекурсивный алгоритм линейного поиска элемента x в неотсортированном массиве A размера N последовательно перебирает его элементы от индекса 0 до $N - 1$, сравнивая каждый элемент A_i с искомым значением. При совпадении поиск завершается с результатом — индексом найденного элемента; при исчерпании массива без совпадений — результатом -1. Схема алгоритма приведена на рисунке 2.1.

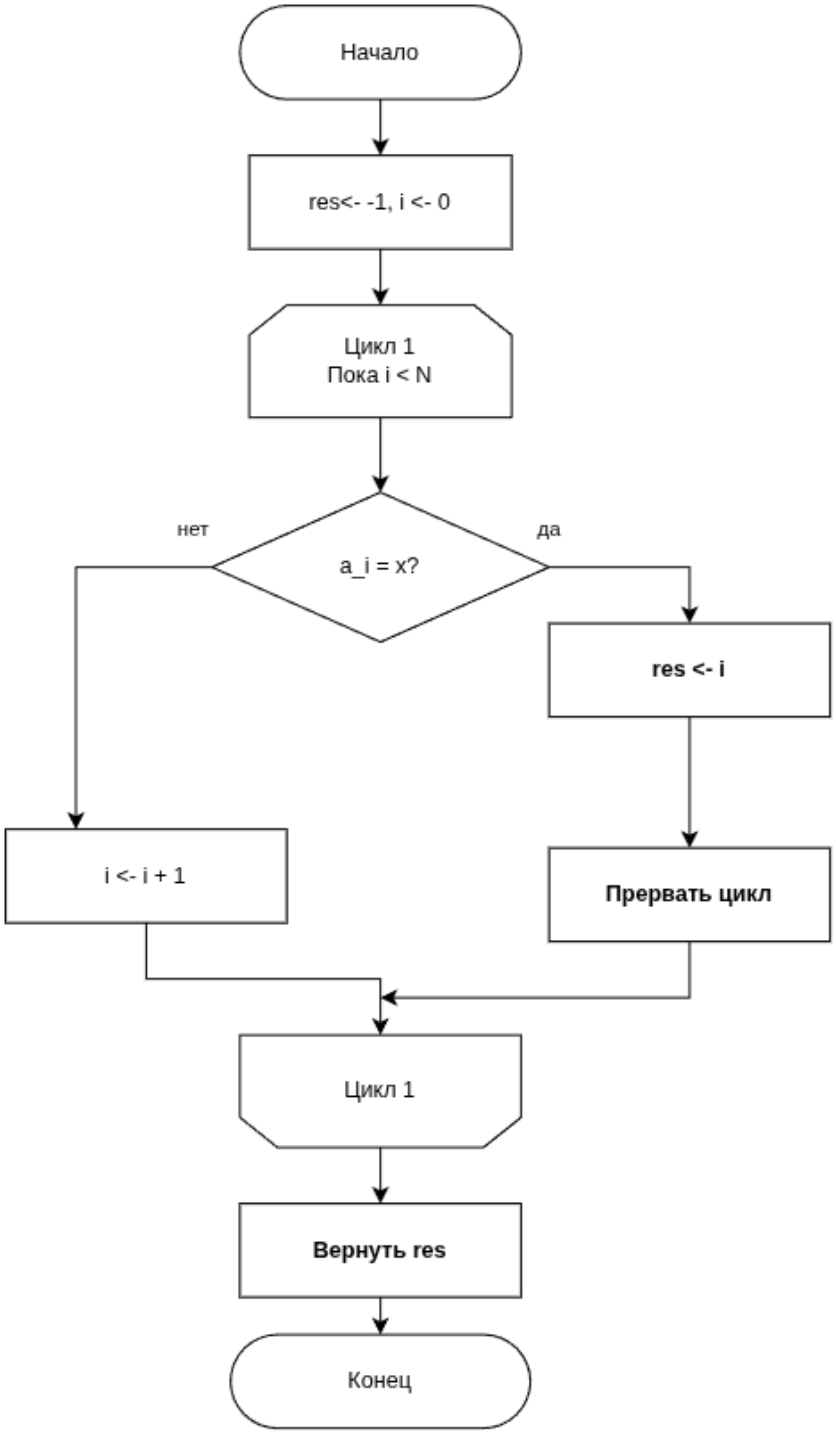


Рисунок 2.1 — Схема алгоритма линейного поиска

2.1.2 Бинарный поиск, нерекурсивная реализация

Итеративный алгоритм бинарного поиска элемента x в отсортированной по возрастанию копии массива A' размера N на каждом шаге цикла делит текущий диапазон $[l; r]$ пополам и сравнивает искомое значение со средним элементом диапазона A'_{mid}, сужая диапазон к одной из половин при несовпадении; цикл завершается при исчерпании диапазона, а в варианте 1 также досрочно

при совпадении. Рассматриваются два варианта реализации, различающиеся организацией сравнений на каждом шаге. В варианте 1 диапазон $[l; r]$ замкнутый: $l = 0$, $r = N - 1$, цикл выполняется, пока $l \leqslant r$; найденный индекс сохраняется в отдельной переменной-результате, а выход происходит через общую точку завершения цикла (схема — на рисунке 2.2). В варианте 2 совпадение со средним элементом не проверяется на каждом шаге. Цикл выполняется, пока $l < r$, и на каждом шаге сравнивается только A'_{mid} с x: при $A'_{mid} < x$ полагается $l = mid + 1$, иначе — $r = mid$. После окончания цикла в найденной позиции l проверяется совпадение $A'_l = x$ (схема — на рисунке 2.3). Таким образом, поиск не завершается досрочно, а при нескольких вхождениях значения возвращается первое из них. Вариант 1 и вариант 2 различаются числом сравнений (см. раздел 4.2), а объём затрачиваемой памяти у них одинаков.

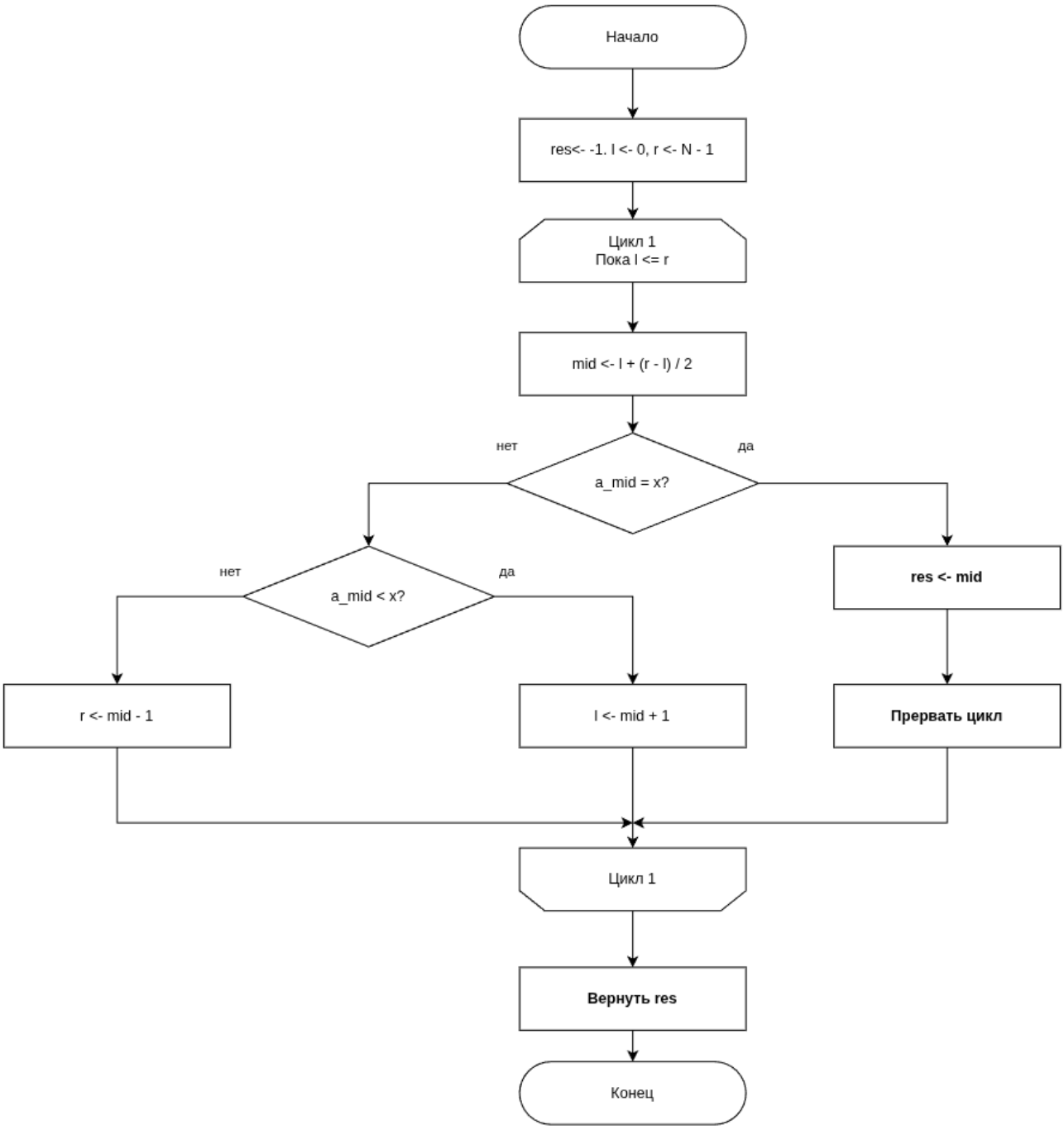


Рисунок 2.2 — Схема алгоритма бинарного поиска, нерекурсивная реализация, вариант 1

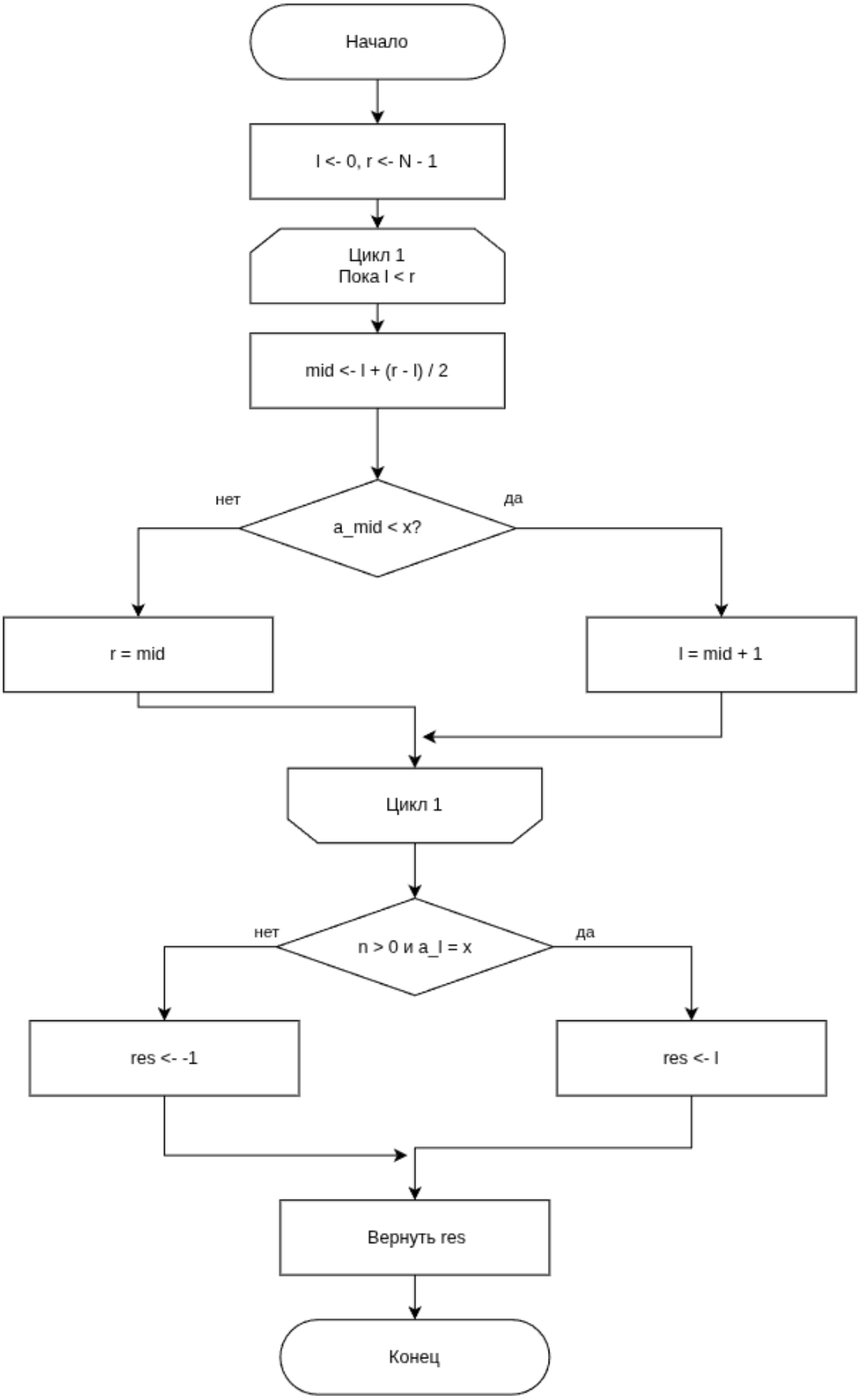


Рисунок 2.3 — Схема алгоритма бинарного поиска, нерекурсивная реализация, вариант 2

2.1.3 Бинарный поиск, рекурсивная реализация

Рекурсивный алгоритм бинарного поиска элемента x в границах текущего диапазона $[l; r]$ отсортированной копии массива A' (в первом вызове $l = 0$, $r = N - 1$) реализует тот же принцип деления диапазона пополам, что и нерекурсивные варианты, но организует повторение самовызовом функции с новыми границами диапазона вместо цикла: при несовпадении искомого значения со средним элементом диапазона A'_{mid} функция вызывает саму себя для соответствующей половины диапазона, передавая новые значения l и r как аргументы.

Принципиальное отличие рекурсивной реализации от нерекурсивных заключается в способе организации повторения: вместо цикла с изменяемыми переменными l и r используется самовызов функции. Именно это отличие определяет разницу в затрачиваемой памяти между реализациями (см. раздел 4.1). Схема алгоритма приведена на рисунке 2.4.

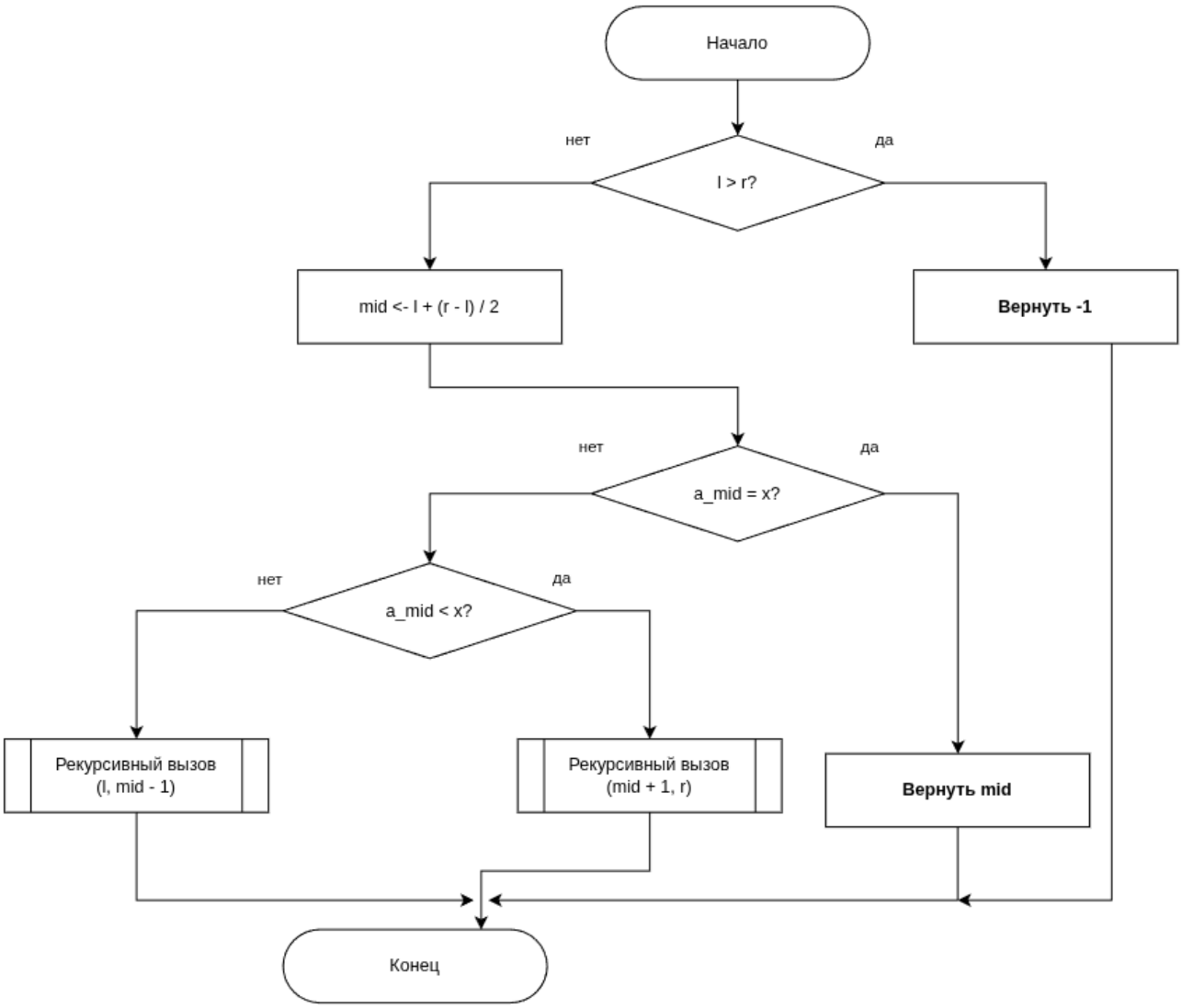


Рисунок 2.4 — Схема алгоритма бинарного поиска, рекурсивная реализация

2.2 Требования к программному обеспечению

Программное обеспечение должно:

— запрашивать у пользователя размер массива N, после чего запрашивать с клавиатуры N целых чисел, формируя массив A;

— создавать дубликат массива A, сортировать его по возрастанию и выводить отсортированный массив на экран;

— циклически выводить меню, посредством которого пользователь сможет ввести искомое значение x и выбрать алгоритм поиска;

— быть реализовано на языке программирования со статической типизацией.

3 Технологическая часть

3.1 Средства реализации

В качестве языка программирования со статической типизацией выбран C++ (стандарт C++17) [2]. Для реализации использованы средства стандартной библиотеки языка:

— `std::vector` — динамический контейнер для хранения массива;

— функции ввода-вывода `scanf`/`printf` стандартной библиотеки C.

Для сборки исполняемых файлов использован компилятор GCC с флагами `-std=c++17 -Wall -Wextra -O2`.

3.2 Реализация алгоритмов

Объявления разработанных функций вынесены в заголовочный файл `algorithms.hpp` (листинг А.1), сами определения размещены в файле `algorithms.cpp` (листинг А.2). Программа с интерактивным меню, использующая эти функции, реализована в файле `main.cpp` (листинг А.3).

3.3 Тестирование реализации алгоритмов

Для проверки корректности реализации разработан модуль тестирования `task_test.cpp` (листинг А.4). План тестирования охватывает: поиск элемента в начале, середине и конце массива; поиск отсутствующего элемента (меньше минимального, больше максимального и внутри диапазона значений); работу с массивом из одного элемента.

Тесты 1–6 выполнены на фиксированном отсортированном массиве $A = \{4, 15, 25, 30, 46, 51, 66, 87, 92, 96\}$.

Таблица 3.1 — Описание тестов

№	Описание теста	Входные данные	Ожидаемый результат	Полученный результат
1	Поиск элемента в начале массива (все 4 реализации)	$x = 4$	Индекс 0	Индекс 0
2	Поиск элемента в конце массива (все 4 реализации)	$x = 96$	Индекс 9	Индекс 9
3	Поиск элемента в середине массива (все 4 реализации)	$x = 51$	Индекс 5	Индекс 5
4	Поиск элемента меньше минимального (все 4 реализации)	$x = -5$	Не найден (-1)	Не найден (-1)
5	Поиск элемента больше максимального (все 4 реализации)	$x = 1000$	Не найден (-1)	Не найден (-1)
6	Поиск отсутствующего элемента внутри диапазона (все 4 реализации)	$x = 50$	Не найден (-1)	Не найден (-1)
7	Поиск в массиве из одного элемента, элемент присутствует	$A = \{42\}$, $x = 42$	Индекс 0	Индекс 0
8	Поиск в массиве из одного элемента, элемент отсутствует	$A = \{42\}$, $x = 7$	Не найден (-1)	Не найден (-1)

Все тесты были успешно пройдены.

4 Исследовательская часть

4.1 Оценка затрачиваемой памяти

Память, затрачиваемая реализацией алгоритма, складывается из памяти на служебные переменные и памяти на кадры стека вызовов функций (для рекурсивной реализации).

Нерекурсивные реализации (линейный поиск и оба варианта бинарного поиска) используют фиксированное число служебных переменных, не зависящее от размера массива N: индексы границ диапазона, середину и переменную-результат, которая хранится на протяжении всего цикла и возвращается через единственную точку выхода. Дополнительная память, затрачиваемая этими реализациями, составляет $O(1)$.

Варианты 1 и 2 бинарного поиска используют одинаковое число переменных, поэтому объём занимаемой памяти у них совпадает.

Бинарный поиск с рекурсивной реализацией на каждый самовызов функции добавляет новый кадр в стек вызовов, хранящий аргументы функции (ссылку на массив, искомое значение x, границы диапазона l и r), адрес возврата и служебную информацию компилятора. Кадр не освобождается до завершения соответствующего вызова, поэтому объём затрачиваемой памяти определяется высотой дерева рекурсивных вызовов h — максимальной глубиной рекурсии [1].

На каждом рекурсивном вызове диапазон поиска $[l; r]$ делится пополам, поэтому дерево вызовов вырождается в цепочку высотой

$$h = \lceil \log_2(N + 1) \rceil ,$$

что совпадает с максимальным числом итераций нерекурсивных реализаций (см. раздел 4.2).

Если обозначить через c объём памяти, затрачиваемый на один кадр стека вызова функции (константная величина, не зависящая от N), то суммарный объём дополнительной памяти, затрачиваемой рекурсивной реализацией в худшем случае, составляет

$$M_{\text{rec}}(N) = c \cdot h = c \cdot \lceil \log_2(N + 1) \rceil = O(\log_2 N).$$

Таким образом, рекурсивная реализация бинарного поиска требует $O(\log_2 N)$ дополнительной памяти против $O(1)$ у нерекурсивных реализаций, решающих ту же задачу.

4.2 Оценка временной эффективности

Временная эффективность алгоритмов оценивается в терминах числа сравнений искомого значения x с элементами массива — операции, определяющей асимптотику алгоритмов [1].

4.2.1 Линейный поиск

— лучший случай: искомый элемент находится на первой проверяемой позиции массива — требуется 1 сравнение, $T_{\min}(N) = 1 = O(1)$;

— худший случай: искомый элемент находится на последней позиции массива либо отсутствует в нём — требуется N сравнений, $T_{\max}(N) = N = O(N)$.

4.2.2 Бинарный поиск

Вариант 1 и рекурсивная реализация делят диапазон пополам с проверкой совпадения на каждом шаге. Шаг, не завершившийся совпадением, требует 2 сравнения элементов с искомым значением (проверка совпадения и проверка порядка), шаг с обнаружением искомого элемента — 1 сравнение.

Вариант 2 выполняет на каждом шаге одно сравнение (проверка порядка), а проверка совпадения выполняется один раз после цикла. Ранний выход при совпадении отсутствует.

Для варианта 1 и рекурсивной реализации лучший случай — искомый элемент совпадает с элементом в середине первоначального диапазона, требуется 1 сравнение: $T_{\min}(N) = 1 = O(1)$. Худший случай — искомый элемент отсутствует в массиве, выполняется $\lceil \log_2(N+1) \rceil$ шагов по 2 сравнения: $T_{\max}(N) = 2\lceil \log_2(N+1) \rceil = O(\log_2 N)$.

Для варианта 2 лучший случай — при $N \geq 1$ требуется $\lfloor \log_2 N \rfloor$ шагов цикла и одна проверка совпадения: $T_{\min}(N) = \lfloor \log_2 N \rfloor + 1 = O(\log_2 N)$. Худший случай — $\lceil \log_2 N \rceil$ шагов цикла и одна проверка совпадения: $T_{\max}(N) = \lceil \log_2 N \rceil + 1 = O(\log_2 N)$.

Так как в варианте 2 минимальное число сравнений логарифмично, его лучший случай асимптотически хуже, чем у варианта 1 и рекурсивной реализации, тогда как худший случай асимптотически совпадает. Число шагов в худшем случае соответствует высоте дерева рекурсивных вызовов h из раздела 4.1.

4.3 Результаты сравнительного анализа

Результаты теоретической оценки ресурсной эффективности рассмотренных алгоритмов и их реализаций сведены в таблицу 4.1.

Таблица 4.1 — Сравнительный анализ ресурсной эффективности реализаций

Реализация	$T_{\min}(N)$	$T_{\max}(N)$	Требования, накладываемые алгоритмом	Доп. память
Линейный поиск (нерекурсивный)	$O(1)$	$O(N)$	—	$O(1)$
Бинарный поиск, нерекурсивный, вариант 1	$O(1)$	$O(\log_2 N)$	массив предварительно отсортирован и поддерживается отсортированным	$O(1)$
Бинарный поиск, нерекурсивный, вариант 2	$O(\log_2 N)$	$O(\log_2 N)$	массив предварительно отсортирован и поддерживается отсортированным	$O(1)$
Бинарный поиск, рекурсивный	$O(1)$	$O(\log_2 N)$	массив предварительно отсортирован и поддерживается отсортированным	$O(\log_2 N)$

Дополнительное требование, налагаемое рекурсивной реализацией и не отражённое в таблице напрямую, — ограничение на глубину рекурсии,

задаваемое размером стека вызовов операционной системы: при достаточно больших N реализация теоретически подвержена риску переполнения стека, тогда как нерекурсивные реализации такого ограничения не имеют.

4.4 Выводы

По результатам сравнительного анализа можно сделать следующие выводы:

— бинарный поиск на отсортированном массиве асимптотически эффективнее по времени линейного поиска на неотсортированном массиве в худшем случае — $O(\log_2 N)$ против $O(N)$ — ценой предварительной сортировки массива;

— вариант 1 и рекурсивная реализация бинарного поиска эквивалентны по временной эффективности, вариант 2 отличается отсутствием раннего выхода: его лучший случай $O(\log_2 N)$ вместо $O(1)$, худший случай асимптотически тот же;

— по ёмкостной эффективности нерекурсивные реализации превосходят рекурсивную: $O(1)$ против $O(\log_2 N)$ дополнительной памяти, затрачиваемой на кадры стека вызовов;

— при прочих равных для решения рассмотренной задачи предпочтительна нерекурсивная реализация бинарного поиска (вариант 1) — она не уступает рекурсивной по времени, не несёт накладных расходов на память, свойственных рекурсии, не подвержена риску переполнения стека вызовов и при совпадении завершается досрочно. Вариант 2 целесообразен там, где важна предсказуемость числа сравнений, не зависящая от значения искомого элемента.

ЗАКЛЮЧЕНИЕ

Цель работы достигнута: выполнен сравнительный анализ ресурсной эффективности алгоритмов.

В ходе проделанной работы были решены поставленные задачи:

— описана решаемая задача согласно варианту;

— разработаны рекурсивный и нерекурсивные алгоритмы решения задачи, согласно варианту;

— разработанные алгоритмы описаны схемами алгоритмов;

— описаны требования к программному обеспечению;

— описаны средства реализации алгоритмов;

— реализованы разработанные алгоритмы;

— выполнено тестирование реализации алгоритмов;

— выполнена теоретическая оценка затрачиваемой реализацией каждого алгоритма памяти (для рекурсивного алгоритма — на материале анализа высоты дерева рекурсивных вызовов и оценки затрачиваемой на один вызов функции памяти);

— оценена временная эффективность алгоритмов или их реализаций в лучшем и худшем случае в терминах числа сравнений искомой величины с другими величинами;

— результаты сравнительного анализа представлены в виде таблицы;

— сделаны выводы из сравнительного анализа реализации разработанных алгоритмов решения одной и той же задачи, заданной вариантом, по критериям временной и ёмкостной эффективности, а также дополнительных требований, налагаемых алгоритмом.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *Skiena S. S.* The Algorithm Design Manual. — 2-е изд. — Springer-Verlag London, 2008. — Гл. 2.2 The Big Oh Notation; 4.9 Binary Search and Related Algorithms. С. 34—36, 132—134. — ISBN 978-1-84800-069-8.
2. ISO/IEC 14882:2017. Programming languages — C++. — International Organization for Standardization, 2017.

ПРИЛОЖЕНИЕ А
Листинги программы

Листинг А.1 — Заголовочный файл с объявлениями функций

```
#pragma once

#include <vector>

using std::vector;

vector<int> read_array(int n);

void sort_arr(vector<int> &arr, int n);

int linear_search(const vector<int> &arr, int n, int x);

int binary_search_iterative_v1(const vector<int> &arr, int n,
   int x);

int binary_search_iterative_v2(const vector<int> &arr, int n,
   int x);

int binary_search_recursive(const vector<int> &arr, int x, int
   l, int r);

void print_arr(const vector<int> &arr);
```

Листинг А.2 — Исходный код с реализацией алгоритмов и вспомогательных функций

```
#include "algorithms.hpp"

#include <algorithm>
#include <cstdio>

vector<int> read_array(int n) {
  vector<int> arr(n);
  printf("Введите %d чисел: ", n);
  for (int i = 0; i < n; i++) {
    int rc = scanf("%d", &arr[i]);
    (void)rc;
  }
```

```
  return arr;
}

void sort_arr(vector<int> &arr, int n) {
  bool swapped;
  for (int i = 0; i < n; i++) {
    swapped = false;
    for (int j = 0; j + 1 < n - i; j++) {
      if (arr[j] > arr[j + 1]) {
        std::swap(arr[j], arr[j + 1]);
        swapped = true;
      }
    }
    if (!swapped)
      break;
  }
}

int linear_search(const vector<int> &arr, int n, int x) {
  int result = -1;
  for (int i = 0; i < n; i++) {
    if (arr[i] == x) {
      result = i;
      break;
    }
  }
  return result;
}

int binary_search_iterative_v1(const vector<int> &arr, int n,
   int x) {
  int result = -1;
  int l = 0;
  int r = n - 1;
  while (l <= r) {
    int mid = l + (r - l) / 2;
    if (arr[mid] == x) {
      result = mid;
      break;
    }
    if (arr[mid] < x)
```

```
      l = mid + 1;
    else
      r = mid - 1;
  }
  return result;
}

int binary_search_iterative_v2(const vector<int> &arr, int n,
   int x) {
  int result = -1;
  int l = 0;
  int r = n - 1;
  while (l < r) {
    int mid = l + (r - l) / 2;
    if (arr[mid] < x)
      l = mid + 1;
    else
      r = mid;
  }
  if (n > 0 && arr[l] == x)
    result = l;
  return result;
}

int binary_search_recursive(const vector<int> &arr, int x, int
   l, int r) {
  if (l > r)
    return -1;
  int mid = l + (r - l) / 2;
  if (arr[mid] == x)
    return mid;
  if (arr[mid] < x)
    return binary_search_recursive(arr, x, mid + 1, r);
  return binary_search_recursive(arr, x, l, mid - 1);
}

void print_arr(const vector<int> &arr) {
  for (int v : arr)
    printf("%d ", v);
  printf("\n");
}
```

Листинг А.3 — Исходный код программы с интерактивным меню

```
#include "algorithms.hpp"

#include <cstdio>

int main() {
  int n;
  printf("Введите размер массива N: ");
  int rc = scanf("%d", &n);
  (void)rc;

  vector<int> arr = read_array(n);

  vector<int> sorted_arr = arr;
  sort_arr(sorted_arr, n);

  printf("Отсортированный массив: ");
  print_arr(sorted_arr);

  int choice;
  do {
    printf("\nМЕНЮ\n");
    printf("1 - линейный поиск\n");
    printf("2 - бинарный поиск, нерекурсивная реализация,
       вариант 1\n");
    printf("3 - бинарный поиск, нерекурсивная реализация,
       вариант 2\n");
    printf("4 - бинарный поиск, рекурсивная реализация\n");
    printf("0 - выход\n");
    printf("Выберите пункт меню: ");
    int rc = scanf("%d", &choice);
    (void)rc;

    if (choice >= 1 && choice <= 4) {
      int x;
      printf("Введите x: ");
      int x_rc = scanf("%d", &x);
      (void)x_rc;
      int index = -1;

      switch (choice) {
```

```
      case 1:
        index = linear_search(arr, n, x);
        break;
      case 2:
        index = binary_search_iterative_v1(sorted_arr, n, x);
        break;
      case 3:
        index = binary_search_iterative_v2(sorted_arr, n, x);
        break;
      case 4:
        index = binary_search_recursive(sorted_arr, x, 0, n - 1);
        break;
      }

      if (index == -1)
        printf("Элемент %d не найден.\n", x);
      else
        printf("Элемент %d найден, индекс: %d.\n", x, index);
    } else if (choice != 0) {
      printf("Некорректный пункт меню.\n");
    }
  } while (choice != 0);

  return 0;
}
```

Листинг А.4 — Исходный код модуля автоматического тестирования

```
#include "algorithms.hpp"

#include <cstdio>

int tests_run = 0;
int tests_failed = 0;

void check(bool condition, const char *description) {
  tests_run++;
  printf("Тест %2d: %-70s %s\n", tests_run, description,
         condition ? "ОК" : "ОШИБКА");
  if (!condition)
    tests_failed++;
}
```

```
int main() {
  const vector<int> sorted_arr = {4, 15, 25, 30, 46, 51, 66, 87,
     92, 96};
  const int n = 10;

  // Поиск существующего элемента в начале массива
  check(linear_search(sorted_arr, n, 4) == 0, "linear_search:
     элемент в начале массива");
  check(binary_search_iterative_v1(sorted_arr, n, 4) == 0,
        "binary_search_iterative_v1: элемент в начале массива");
  check(binary_search_iterative_v2(sorted_arr, n, 4) == 0,
        "binary_search_iterative_v2: элемент в начале массива");
  check(binary_search_recursive(sorted_arr, 4, 0, n - 1) == 0,
        "binary_search_recursive: элемент в начале массива");

  // Поиск существующего элемента в конце массива
  check(linear_search(sorted_arr, n, 96) == 9, "linear_search:
     элемент в конце массива");
  check(binary_search_iterative_v1(sorted_arr, n, 96) == 9,
        "binary_search_iterative_v1: элемент в конце массива");
  check(binary_search_iterative_v2(sorted_arr, n, 96) == 9,
        "binary_search_iterative_v2: элемент в конце массива");
  check(binary_search_recursive(sorted_arr, 96, 0, n - 1) == 9,
        "binary_search_recursive: элемент в конце массива");

  // Поиск существующего элемента в середине массива
  check(linear_search(sorted_arr, n, 51) == 5, "linear_search:
     элемент в середине массива");
  check(binary_search_iterative_v1(sorted_arr, n, 51) == 5,
        "binary_search_iterative_v1: элемент в середине
           массива");
  check(binary_search_iterative_v2(sorted_arr, n, 51) == 5,
        "binary_search_iterative_v2: элемент в середине
           массива");
  check(binary_search_recursive(sorted_arr, 51, 0, n - 1) == 5,
        "binary_search_recursive: элемент в середине массива");

  // Поиск отсутствующего элемента меньше минимального
  check(linear_search(sorted_arr, n, -5) == -1, "linear_search:
     элемент меньше минимального");
  check(binary_search_iterative_v1(sorted_arr, n, -5) == -1,
```

```
      "binary_search_iterative_v1: элемент меньше
         минимального");
check(binary_search_iterative_v2(sorted_arr, n, -5) == -1,
      "binary_search_iterative_v2: элемент меньше
         минимального");
check(binary_search_recursive(sorted_arr, -5, 0, n - 1) == -1,
      "binary_search_recursive: элемент меньше минимального");

// Поиск отсутствующего элемента больше максимального
check(linear_search(sorted_arr, n, 1000) == -1,
   "linear_search: элемент больше максимального");
check(binary_search_iterative_v1(sorted_arr, n, 1000) == -1,
      "binary_search_iterative_v1: элемент больше
         максимального");
check(binary_search_iterative_v2(sorted_arr, n, 1000) == -1,
      "binary_search_iterative_v2: элемент больше
         максимального");
check(binary_search_recursive(sorted_arr, 1000, 0, n - 1) ==
   -1,
      "binary_search_recursive: элемент больше максимального");

// Поиск отсутствующего элемента внутри диапазона
check(linear_search(sorted_arr, n, 50) == -1, "linear_search:
   отсутствующий элемент внутри диапазона");
check(binary_search_iterative_v1(sorted_arr, n, 50) == -1,
      "binary_search_iterative_v1: отсутствующий элемент
         внутри диапазона");
check(binary_search_iterative_v2(sorted_arr, n, 50) == -1,
      "binary_search_iterative_v2: отсутствующий элемент
         внутри диапазона");
check(binary_search_recursive(sorted_arr, 50, 0, n - 1) == -1,
      "binary_search_recursive: отсутствующий элемент внутри
         диапазона");

// Массив из одного элемента
const vector<int> single_arr = {42};
check(linear_search(single_arr, 1, 42) == 0, "linear_search:
   массив из одного элемента, найден");
check(binary_search_iterative_v1(single_arr, 1, 42) == 0,
      "binary_search_iterative_v1: массив из одного элемента,
         найден");
```

```
  check(binary_search_iterative_v2(single_arr, 1, 42) == 0,
        "binary_search_iterative_v2: массив из одного элемента,
           найден");
  check(binary_search_recursive(single_arr, 42, 0, 0) == 0,
        "binary_search_recursive: массив из одного элемента,
           найден");
  check(linear_search(single_arr, 1, 7) == -1, "linear_search:
     массив из одного элемента, не найден");
  check(binary_search_iterative_v1(single_arr, 1, 7) == -1,
        "binary_search_iterative_v1: массив из одного элемента,
           не найден");
  check(binary_search_iterative_v2(single_arr, 1, 7) == -1,
        "binary_search_iterative_v2: массив из одного элемента,
           не найден");
  check(binary_search_recursive(single_arr, 7, 0, 0) == -1,
        "binary_search_recursive: массив из одного элемента, не
           найден");

  printf("\n");
  if (tests_failed == 0)
    printf("Все тесты пройдены успешно (%d/%d).\n", tests_run,
       tests_run);
  else
    printf("Провалено тестов: %d из %d.\n", tests_failed,
       tests_run);

  return tests_failed == 0 ? 0 : 1;
}
```